

Data Base Systems

Data : The data is the simply raw, unorganized facts, figures, symbols, that haven't processed yet. The data doesn't give much information.

Database : A database is a structured, organized collection of related data. In a relational database, the data is organized into rows and columns.

Database Management System : A DBMS is the actual software program that let users and apps interact with the database.

We need DBMS to create database, protect it, retrieve data from it and update it.

"The CRUD operations : Create, Read, Update, Delete"

The major reason we use DBMS is "Data independence". This means the way the data is physically stored on a hard drive is completely separated from how we query it as a user.

Definition of DBMS :

This describes the entire lifecycle of how DBMS handles a data.

1. Defining a database :

Before storing a data, we are telling the DBMS to specify the data type, structure and constraints. We are building the Skeleton of Database.

2. constructing a database:

Once the blueprint is defined, constructing is the actual process of storing the data in a physical storage device. It takes the raw rows as inputs and DBMS organizes them according to our definition.

3. Manipulating a database: once the data is inside, it shouldn't just

Sit there. Manipulating means interacting with the data. This includes querying (searching for specific info) and updating it.

4. Sharing a database: A database is rarely used by one person at a time. sharing means allowing multiple users and programs to access the database simultaneously.

Types of Database:

1. Traditional database: This DB used for basic operations. They deal strictly with structured, alphanumeric data (Text,

Numbers, dates and currently).

The data is highly structured,
cleanly fits into the table (ROWS
and columns), requires less storage.

2. Multimedia database: A multimedia database handles audio, video, images, graphics and animations. The data files are massive. The database has to handle special storage type BLOBs (Binary Large Objects) and they need to compress files to load instantly when requested.

3. Geographic Information Systems/
Spatial Database: It is built

to store and analyze geospatial data
- information tied to physical location
on earth. They store like latitude,
longitude, elevations. It relies on
the specialized query methods.

Feature	File System	v/s	Database Management System (DBMS)
Data Structure	Unstructured or simple files (e.g., .txt , .csv) stored in folders.		Highly structured data organized into tables with defined relationships.
Data Redundancy	High. The same data is often duplicated in multiple files, wasting space.		Low. Normalization rules minimize duplication by linking tables together.
Data Consistency	Poor. If a customer changes their phone number, you have to manually update every single file it appears in.		High. Because data is stored in one place, updating it once updates it everywhere instantly.
Data Independence	No. The data format is tied to the application program. If you change the file structure, the code breaks.		Yes. The structure of the database can be changed without breaking the applications using it.
Search & Querying	Difficult. Requires writing custom code to open, read, and scan through files line-by-line.		Easy. Uses standardized, powerful query languages like SQL to find data in milliseconds.
Concurrent Access	Limited. Usually, only one user or program can edit a file at a time, or they overwrite each other.		Advanced. Allows thousands of users to safely read and write data at the exact same time.
Security & Backup	Basic. Folder-level passwords or file permissions only. Hard to back up while in use.		Robust. Granular security (restrict access down to specific columns) with automated recovery tools.

Characteristics of DBMS :

1. Self - Describing nature (catalog) :

A traditional file system just contains raw data, and the program reading it has to guess. A DBMS is self-describing because it contains not only the data itself but the definition of the data. It stores the information in a data dictionary or Metadata.

2. Insulation between programs and Data (Data independence) :

This separates the database from the application programs. We can alter the table layouts or the data

without modifying the software programs accessing the data.

3. Support of Multiple views of the

data: A database contains massive amount of data. But not every user needs to see all the data so DBMS allows us to create different "views" of the same data for different users.

4. Sharing of data and multi-user

Transaction processing: A DBMS allows multiple users to access the same data at a same time. DBMS uses a concept called "concurrency control".

It helps the DBMS to process data in Sequence.

5. Controlled Redundancy: DBMS

Integrates the data so that each piece of data ideally exists in only one place. DBMS automatically controls the duplication and updates all copies to keep them identical.

6. Security and Authorization

Restrictions: A DBMS has dedicated security subsystem. We can restrict the access to row, column or table.

The ultimate DBMS characteristic -

ACID Test :

Whenever data is modified, a true DBMS guarantees ACID properties :

A - Atomicity : All or Nothing - If a bank transfer fails, the whole transaction rolled back so money will not vanish into thin air.

C - Consistency : The database only moves from one valid to another state, following all predefined rules.

I - Isolation : Concurrent transactions execution happens separately without interfering with each other.

D - Durability : Once a transaction is saved, it stayed saved, even if the power cuts a second later.

Actors on the scene

It refers to the people who are directly involved in the day-to-day operation, design, use of database itself.

The actors are classified into 4 types based on how they interact with the System:

i. **Database Administrators (DBA)**:

→ The DBA is the ultimate guardian of the database. They are responsible for managing the entire system.

- They can grant permission and access rights to the users.
- Monitoring system performance.
- Setting up backup and disaster recovery plans.

ii. Database designers: Designers are the architects. Before a database is built, designers think about what data needs to be stored, what should be constraint, how the structure should be.

- They create Entity-Relationship (ER) diagrams.

→ They define the structure of table, Views and relationship.

iii. System analyst and application programmers :

→ These are Software developers who build the application that the naïve end users interact with.

→ System Analyst: Determine what the end users need from the software application.

→ Application programmer: Write the code (python, java, C++) and embedded SQL queries to fetch, update or delete data from database.

iv. End users :

→ These people use the database for daily tasks. They don't care how the database are built or works.

i. **Naive / parametric End users :** They interact with the database through simple and pre-built applications. This database is accessed frequently.

ii. **Casual End users :** They access the database occasionally, but they write unique SQL queries to get specific information each time.

iii. **Sophisticated / Atypical users :**
Engineers / scientist / Business analyst

who write the complex queries to analyze the data deeply. This users can access the database that is restricted to the other people (public).

iv. Standalone / Independent user:

→ A standalone user maintains database all by themselves. They are the designers, administrator and End user.

→ The database sits on their local computer or phone and it is rarely accessed by others.

→ These types are used for the personal project / Group project.

Workers Behind the Scene

These people care about making the DBMS software engine faster, safer and smarter.

i. DBMS system designers and implementers :

These are the core software engineers who write the codes to create the DBMS software. They write the codes in C language that determines how the database interact with computer hardware.

ii. Tool developers :

→ A DBMS engine itself is a command-line tool. The tool developers build the helpful software packages that makes the

Interaction with database easier for the
"Actors on the Scene".

→ They build the graphical interface,
ER diagram tools, database design
assistants, performance monitors.

iii. operators and maintenance personal :

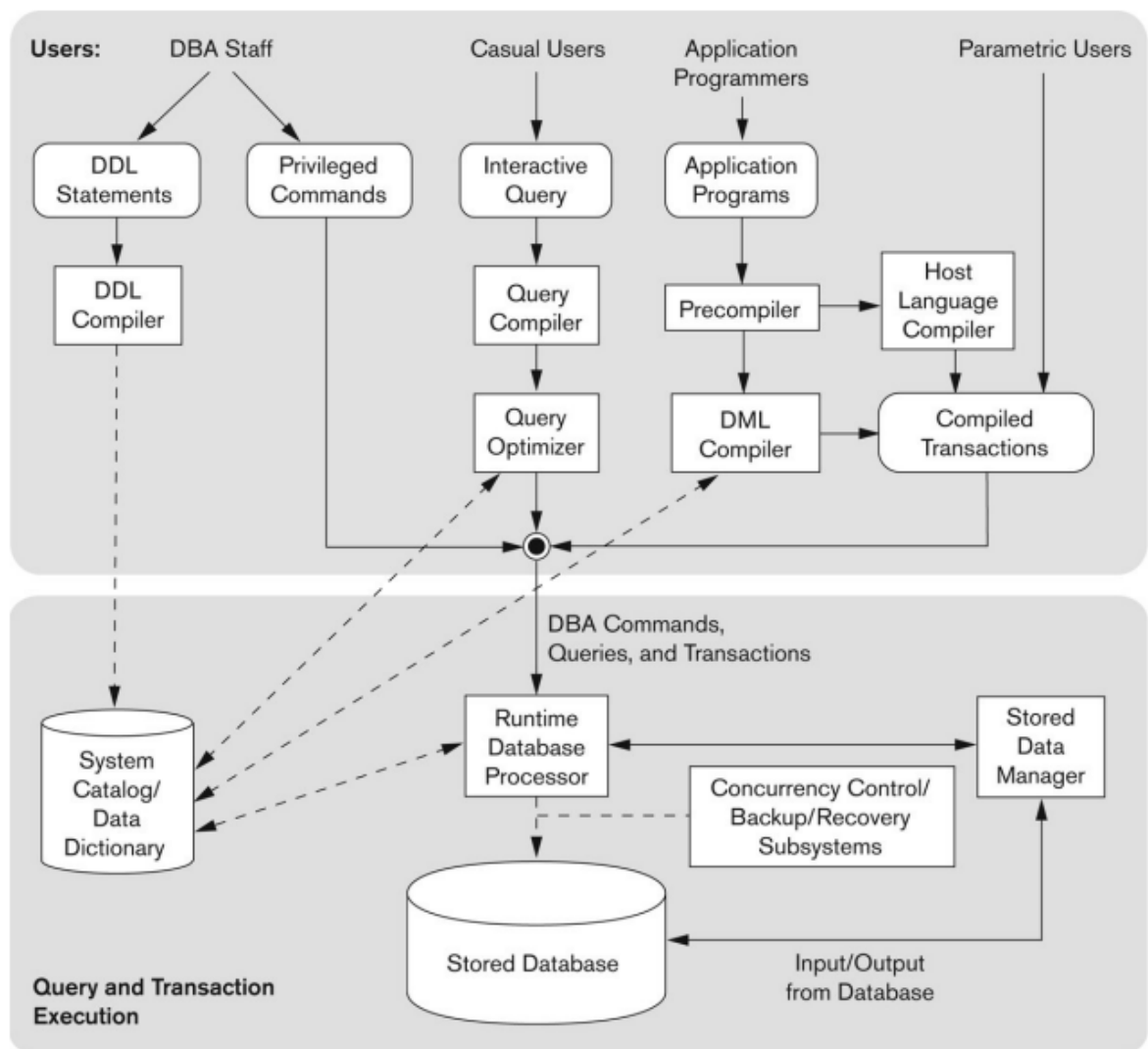
→ These are the system administrators
and data center technicians who keep the
physical hardware running.

→ They manage the physical servers,
rows of hard drives, cooling system and
power supply. If any physical system fails,
these people will repair or replace it.

Database System Environment :

The DB environment is an entire ecosystem that makes the database work.

It is the combination of machine, people, software and rules working together to store and manage the information.



This shows what happens from the moment user types a command to the moment data fetched from hard drive.

The users and Translators :

Computers don't understand human requests, so each type of user goes through specific translator module.

i. DBA staff (The administrator):

→ They build the structure of database and handles security.

→ They write DDL commands (like creating a table)

→ DDL compiler processes these statements and saves the table structure

directly into data dictionary / system catalog.

→ They also issue privileged commands (like creating user accounts) that go straight to the database core system.

ii. Casual users (The searchers):

→ These users typing a direct, manual query into the system.

→ They type an interactive query

→ The query compiler checks for the syntax error.

→ The Query optimizer checks the System Catalog to find the fastest and

Smartest way to search millions of records so the system doesn't lag.

iii. Application programmers (coders):

→ Software developers connects the codes (like C++, python, Java) to the database.

→ They write application programs with database commands.

→ The precompiler splits the code apart. It sends the regular code to Host language compiler (like a Java compiler) and sends the database commands to the DML compiler.

iv. Parametric users (Everyday users):

Everyday people using an app like Instagram viewing the reels or sharing the posts, they just click a button on the screen which triggers the pre-written, Compiler Transactions instantly.

The Execution Core:

once all the user request are translated, they hit the center point and enter the execution core.

v. Runtime database processor

(The engine / The brain):

This is the central command center of the DBMS. It takes all commands, queries, transactions talks to the system

catalog to verify permissions and executes them.

vi. Concurrency control and Backup /

Recovery (The Security Guards) :

→ This module runs closely with the runtime processor.

→ Concurrency control ensures that if two users try to change the same piece of data at the same time, they don't overwrite or corrupt each others work.

→ Backup / Recovery handles saves states so the data wouldn't be lost if the power cuts off suddenly.

vii. Stored Data Manager and Stored Database (The Hands and the Storage) :

Stored Data Manager:

The lowest level Software module.

It handles the actual physical work of reading or writing blocks of data.

Stored Database :

The actual physical Hard drives or cloud where the file lives. The manager retrieves the data from Storage and passes it to the user through I/O Operations.

Database System Architecture :

1. Logical View Architecture :

How the data is organized conceptually inside the System.

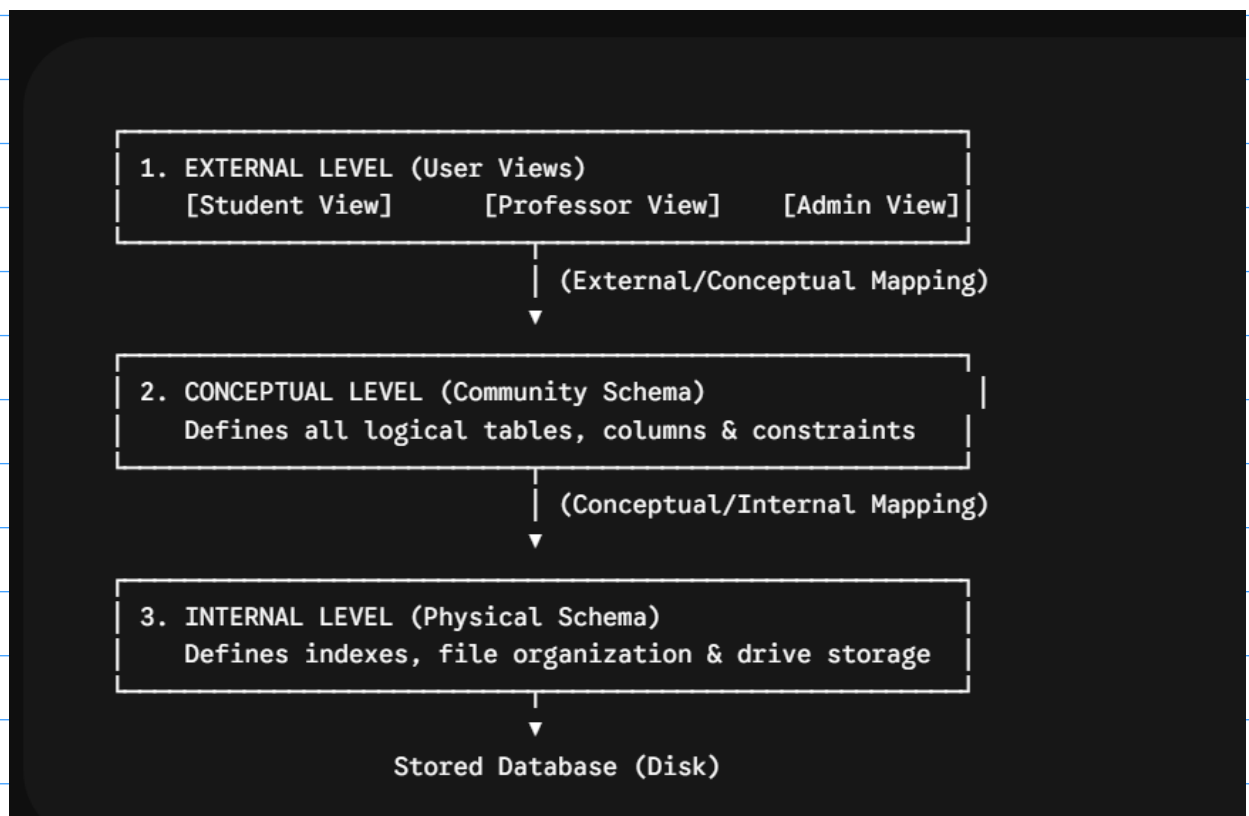
Three - schema Architecture :

→ A conceptual 3-layer framework

(External, conceptual and Internal

levels) used to separate user's view of data from physical database.

→ This ensures Data independence meaning we can modify database without crashing application software.



2. Network and processing Architecture!

How the user's Computer and the database computer talk and share the workload.

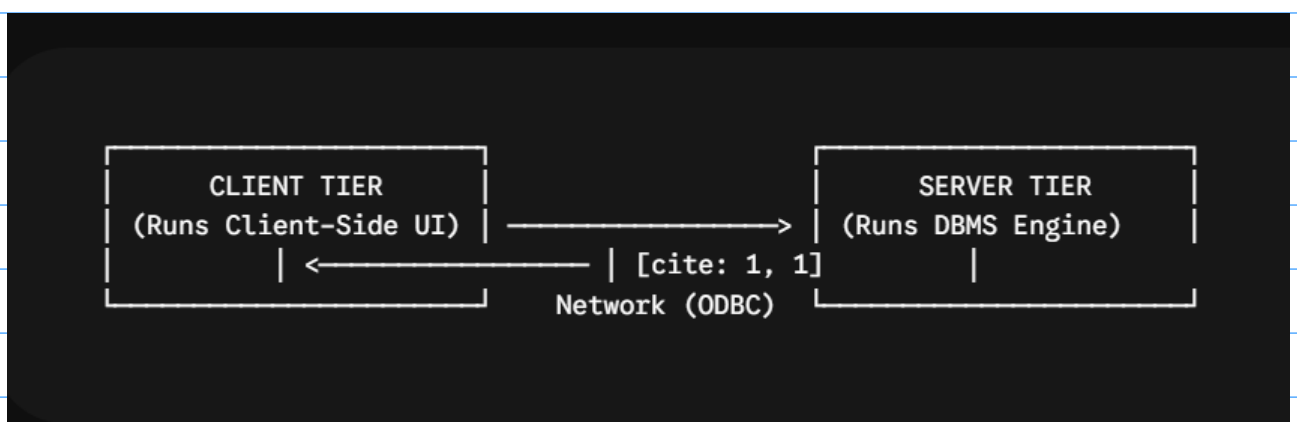
i. Client-server architecture:

A network model where the workload split between client

(user devices requesting data) and
servers (heavy-duty computers
supplying data and services)

ii. Two-tier (2 tier architecture) :

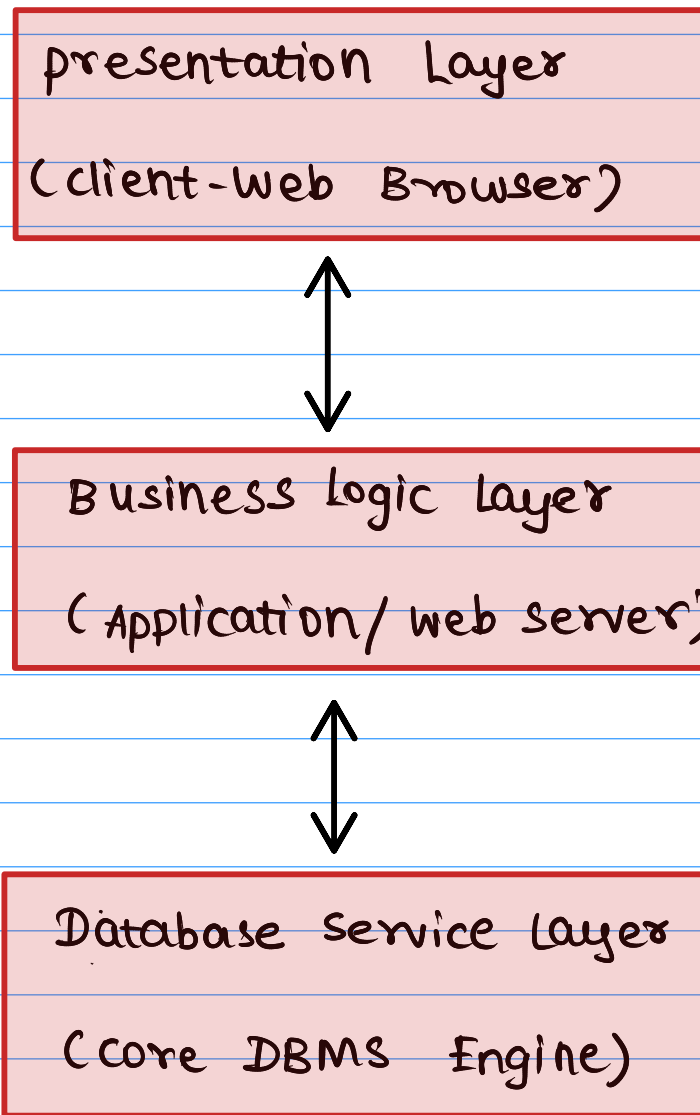
A direct client-server connection
exactly two layers. The client machines
run the user-interface and application
programs and communicate directly with
the database drivers like ODBC.



iii. Three-tier (3 tier Architecture) :

→ The standard model for web application that introduces a middle layer called Application or web server.

→ The client Browser (presentation layer) talks to the web server (Business logic layer) which handles processing and fetches data safely from the deep Database server (Database service layer).



iv.

client-side (client-side Modules)

The dedicated software modules installed directly on user's computer.

The client-side software is responsible for the rendering Graphical user Interface.

handling user inputs and managing network connections.

3. physical Deployment Architecture :

Where the physical database Computers and files located Geographically.

i. Centralized Architecture :

Everything - The database engine Software, data storage files, application processing is packed into a single, central Computer system. Even if the users connect via remote access, all computing is executed at the central computer.

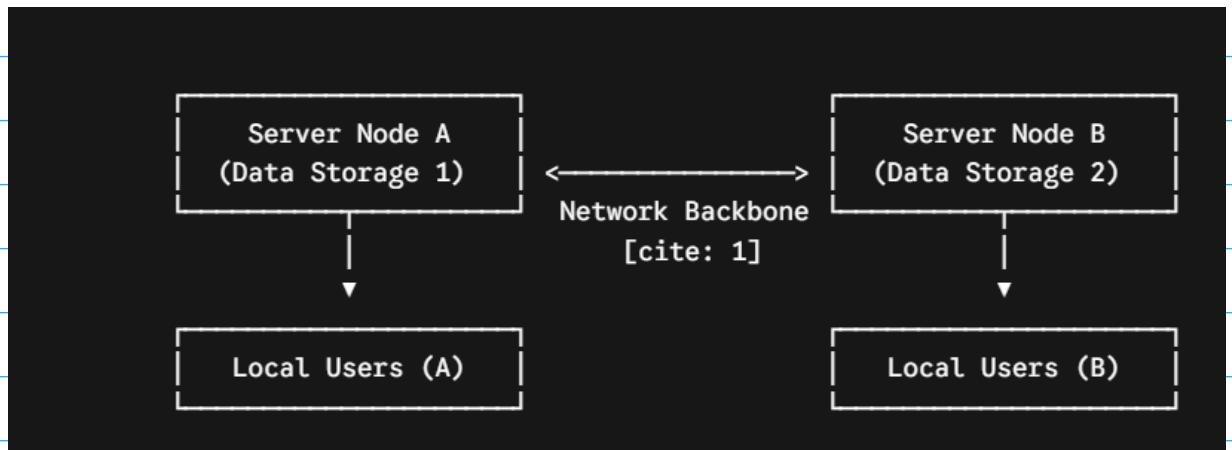


ii. Distributed Architecture :

→ The direct opposite of centralized architecture. The data files and database software are physically broken up and scattered across multiple computers in different physical location, all connected in network.

→ Modern distributed databases have largely evolved into complex client-server based systems, where

a grid of database server supports
a vast network of clients.



Logical Structure Of DB :

- i. It describes about the logical Structure of Database which is used to perform operations like INSERT, DELETE, MODIFY.
- ii. It also tells how the data are Connected, how the data will be processed and how it will be stored.

The core pillars of the logical structure,

i. Data Models

ii. Instance

iii. Schema

1. Data model (Rule & Style) :

i. The data model is the broad conceptual framework or style of the database. It provides the tools and rules for describing the data, relationships and constraints.

ii. The most common model is Relational model. With this model, we can simply represent the data in tables, rows and columns instead of web of connected

nodes (Network model) or hierarchy
of nodes (Hierarchy model).

a. Hierarchy model (The Family-Tree
model):

→ The data is organized into
tree-like structure. It uses parent-child
Relationship. A parent can have multiple
children but a child can have only
one parent.

→ It cannot handle multiple connections.

Ex. If a child wants to connect many
parents, the model broke down.

Department

No	Name
----	------

course

No	Name	units
----	------	-------

student

ID	Name	course
----	------	--------

professor

ID	Name
----	------

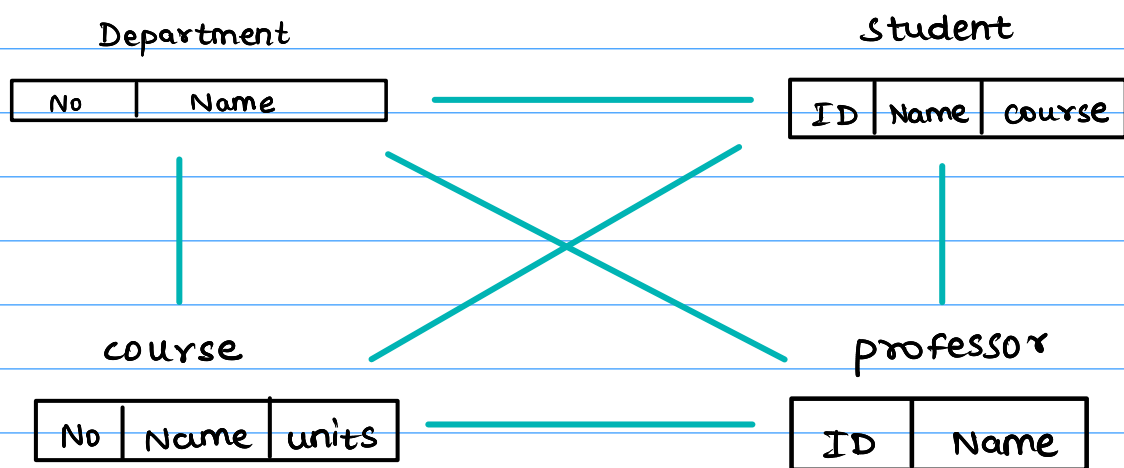
b. Network-model (The spider-web model) :

→ Created to fix the strictness of Hierarchy model

→ It allows a child to have multiple parents. This created a complex

web of intersecting arrows.

→ It depends on pointer (memory address) to link data. If we want modify the structure, then we need the rewrite the entire program.



c. Relational model (The table) :

→ This organizes the data into simple, flat tables with rows and columns.

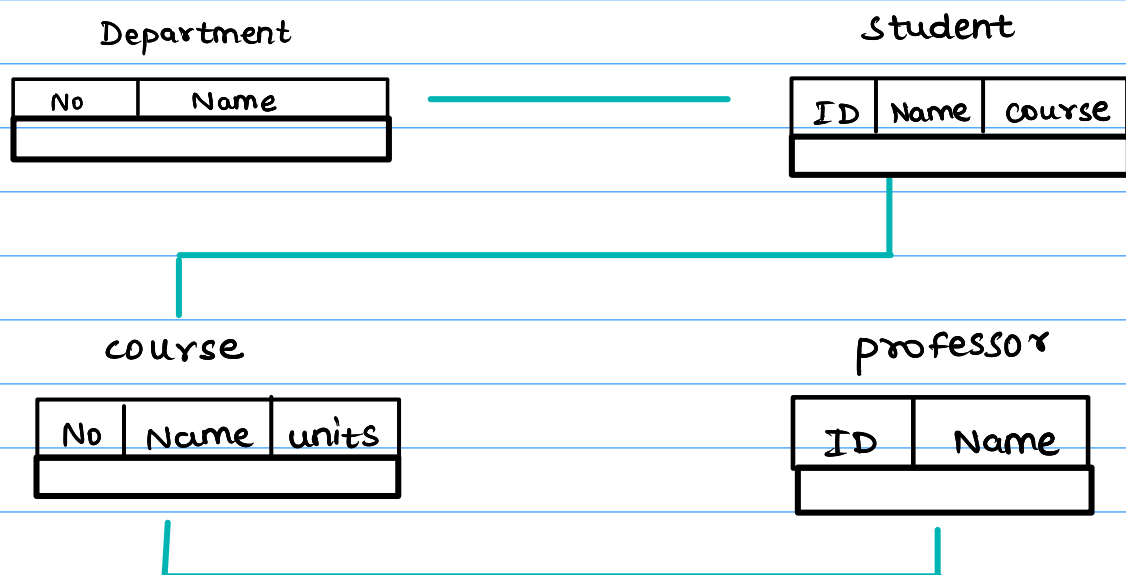
→ It completely threw away pointer

Concept to connect the data. It

Connects data through matching

Values (primary key and foreign key).

→ This model made to modern database easy to use.



The new Empty Table Says - No fixed

physical size, Dynamic tuple insertions,

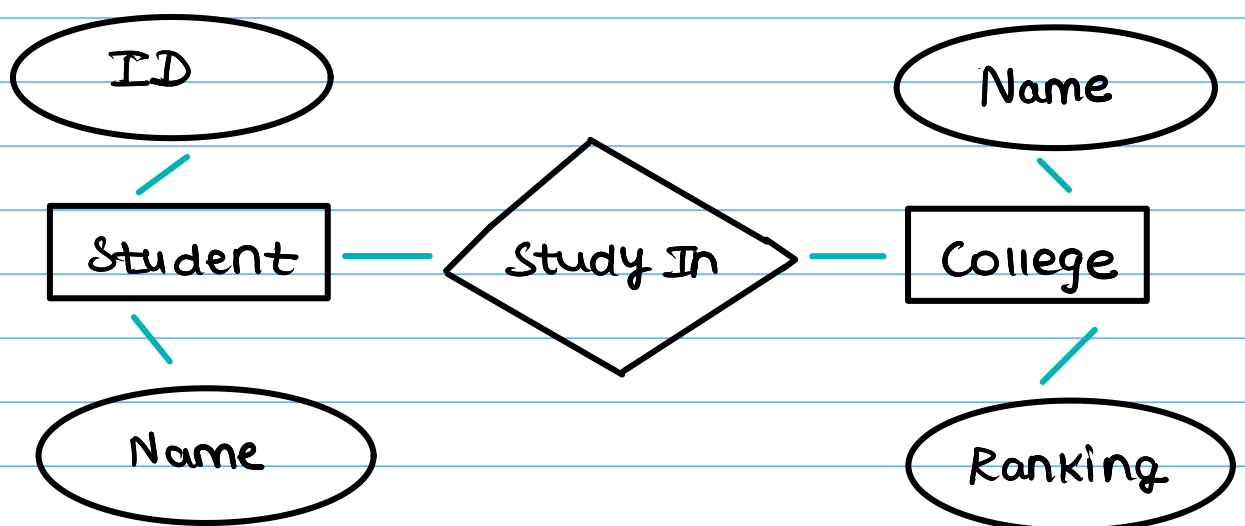
Introduction of NULL.

d. ER Model (The Conceptual Blueprint):

→ The Entity - Relationship model is an unique model used for designing a database before building it in the system.

→ It represents data Visually using diagrams (Boxes for entities, Diamond for relationship).

→ It shows how entities interact.



The Schema :

- i. High level definition / description of data is called Schema.
- ii. This is the permanent structure. It is defined during the database design and changes very rarely.
- iii. It specifies row names, Column names, data types, constraints.

The Instances :

- i. This is the actual filling of the data into the database.
- ii. It changes constantly as rows are added, updated and deleted.

The phases of the database design

cycle:

i. Requirement collection & Analysis :

→ Before drawing anything, we talk to the users, we ask clients their requirement about what data they need to track.

→ A requirement specification document

It states in plain English like we need to store Student's name, age, CGPA and to track what course they take.

ii. Conceptual Design :

→ we take those requirements in plain English and create a

Completely Software-Independent
blueprint. This is where we draw
Entity - Relationship diagrams (ER).

→ A visual chart with boxes (Entity),
Diamond shaped (Relationship). It
does not care whether we will use
Oracle, MySQL, Microsoft.

iii. Logical Design:

→ we map our abstract ER diagram
into a specific type of data model
usually Relational model.

→ We will get the logical schema. We
translate our visual ER diagram to
actual columns, tables, data types

and primary/foreign keys. The table with empty rows defining there is no fixed size/dynamic tuple.

iv. physical Design:

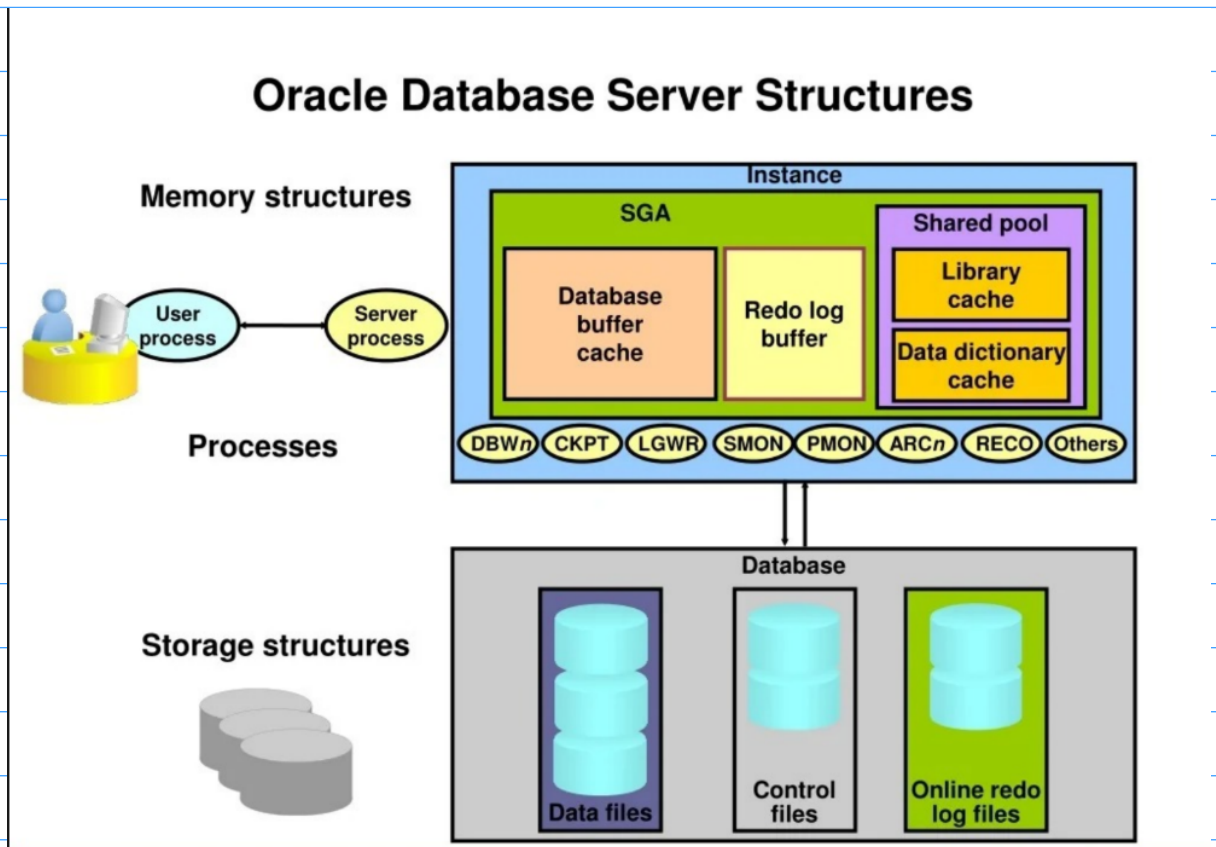
- Finally we adapt the logical model to a specific Database Management System (DBMS) like postgres SQL or SQL server and decides how it will live in the hardware.
- This is Internal Schema. we create the actual SQL code, set up security access, design physical indexes so that queries run faster on server's hard drives.

Oracle DataBase Structure

- i. Oracle database is an Object-Related database management system.
- ii. It consists of instance and data.
- iii. The Oracle database (physical files) is well distinguished from Oracle instance (memory and process).

Together they form the Oracle DB Server, built on three pillars :

- i. Memory Structures
- ii. Background process
- iii. Storage Structures



1. Memory Structures (The instance) :

It is primarily the **SGA** (System Global Area) which is the Shared memory region holding data buffers, SQL caches, log buffers and the **PGA** (Program Global Area) which is the private memory allocated to each session.

Database Buffer cache: Holds the copy of data blocks read from data files shared by all concurrent users.

Redo Log Buffer: A circular buffer holding information / redo entries about the changes made in the database.

Shared pool: Contains the library cache (Shared SQL area), Data dictionary cache and control structures.

Large pool: provides large memory allocation for shared memory session, I/O process and backup operations.

Java and Stream pool: used for JVM session - specific code and Oracle streams queue message buffering respectively.

ii. Background process (The instance):

The Operational engines (like DBWn, LGWR...) that manage memory, write data to disk, handle system maintenance.

DBWn (Database writer): periodically and Asynchronously writes modified (dirty) buffers from the buffer cache to physical data files on disk.

LGWR (Log writer): Write the redo log buffer to disk sequentially when the user commits, when the buffer is $\frac{1}{3}$ full or before database writer writes.

CKPT (checkpoint): Signals data flushing and records checkpoint tracking metadata inside the control file and each data file header.

SMON (System Monitor): Automatically performs crash/instance recovery during startup and cleans up unused temporary segments.

pmON (process Monitor): performs process recovery when a user process fails, cleaning the buffer cache, releasing resources, registering services with Listeners.

Recoverer process (RECO): It automatically connects to the other databases involved in an "in-doubt" distributed transactions and automatically resolves the transaction.

Archiving process (ARCn): Archiving is Oracle's mechanism to preserve transaction history so that the DB can be recovered if the physical disk fails.

iii. Storage Structures (The Database):

It is the physical and logical storage of the database. The actual files living in the disk OS (Data files, Redo log files, Control files).

Logical hierarchy: Database → Schema
→ Tablespace → segment → Extent
→ Oracle data block.

Tablespace: A logical container
(mandatory SYSTEM for core dictionary tables and auxiliary SYS AUX tablespaces).

Segment / Extent / Block: Segments are the collection of extents. Extents are the collection of datablocks which is

the smallest logical unit.

physical mapping rules: Tablespace

might consist of one or more physical data files on disk. But a single data file belongs to only one tablespace.

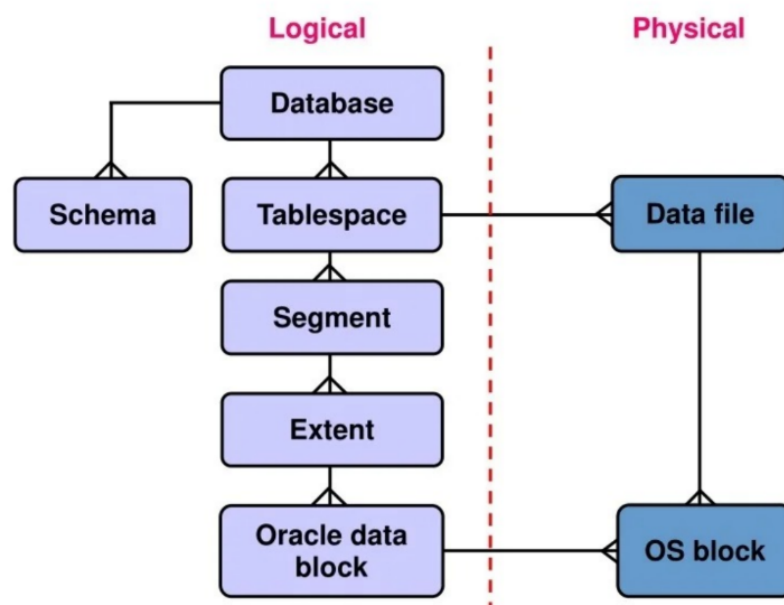
The Key files are Control files

(structure metadata), online Redo

log files (Transaction history)

and backup files.

Logical and Physical Database Structures



Connection and Interaction process:

Connection : The physical communication path established between user process and oracle instance.

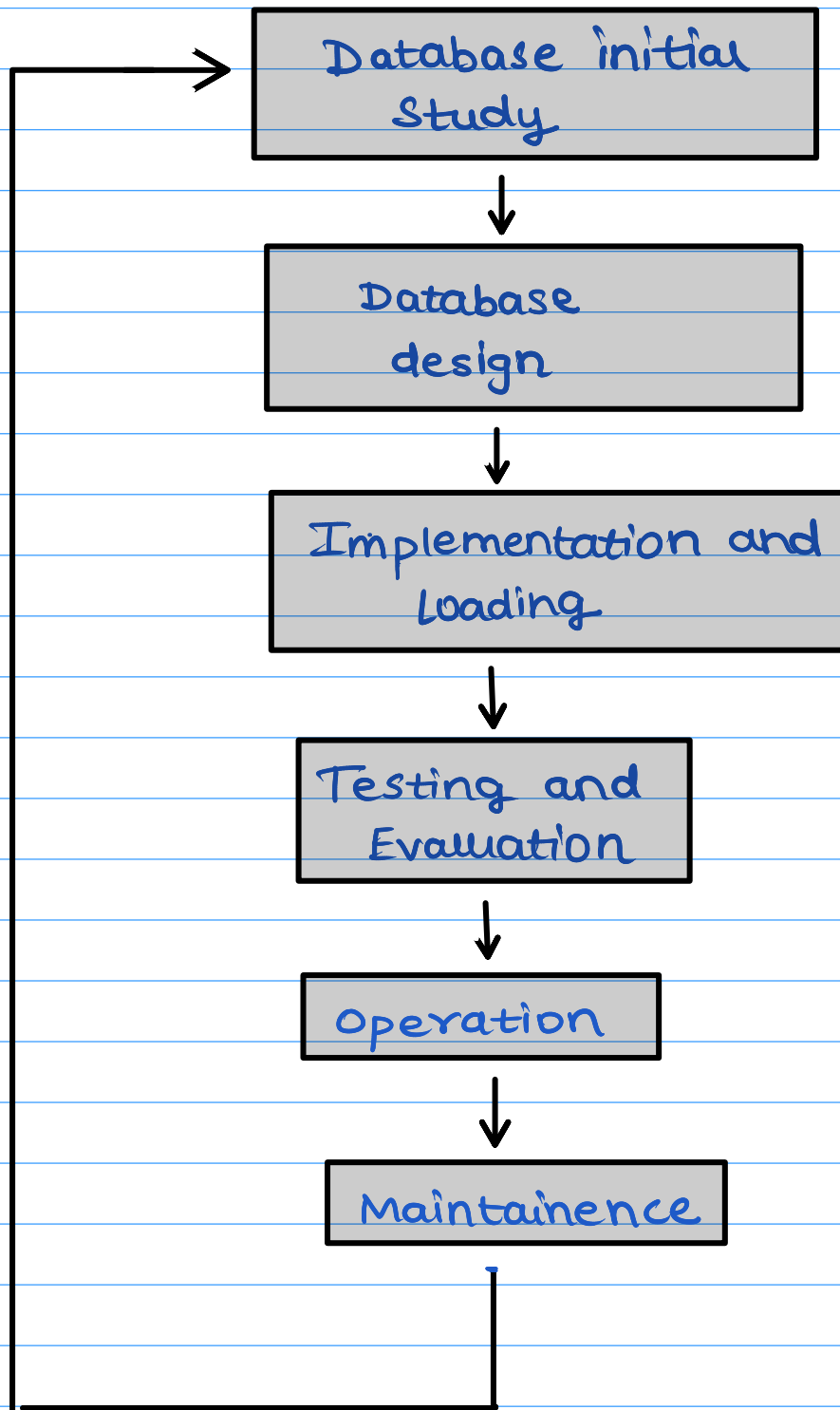
Session: A specific logical connection of a user to an instance created when an user authenticates through user process.

User process : Starts when a user or batch process connected to the database.

Server process: Starts when the user establishes the session, connecting directly to oracle instance to execute client's request.

Module 2

Lifecycle of Database Design:



i. Database Initial Study :

Analyzing the existing system to define business problems, define project Objectives, Set System boundaries.

ii. Database design :

Creating the database structure based on the requirements (ER-diagram, Normalized tables, storage specs).

iii. Implementing and loading :

Installing the database software, building the actual tables using DDL commands and populating with initial data.

iv. Testing and Evaluation:

Testing the system for security, performance, data accuracy, user satisfaction before going live or deploy.

v. Operation:

Launching the system into production so users can run daily transactions. (like querying, inserting, updating data)

vi. Maintenance:

Managing performance tuning, taking regular backups, and making adjustments as the requirement changes.

Entity - Relationship model (ER-model) :

The Entity Relationship model or ER model is defined as the high-level Conceptual data model used to Construct a detailed blueprint of an organization's data.

Entity type : An object that ER-model representing which are identified that having an independent existence.

Ex: Faculty, customer, Student

Entity set : The collection of entities all having the same properties in the Entity - Relationship model.

Attributes : An attribute is the property of an entity type or relationship type.

Attributes holds the actual values that describe an entity.

Based on structure, Multiplicity, Value derivation and keys, Attributes are Classified into :

Simple V/s Composite Attributes :

Simple / Atomic: An attribute composed of a single component with an independent existence. It cannot be divided into smaller sub-components.

Ex: salary, position.

Composite : An attribute composed of multiple components, each with independent existence.

Ex: Address can be split into street, city, District.

Single-valued v/s multi-valued attributes :

Single-valued : An attribute that holds a single value for a single entity occurrence.

Ex: Date of Birth.

Multi-valued : An attribute that holds multiple values for a single entity occurrence.

Ex: A person can have multiple mobile numbers.

Stored v/s Derived attributes :

Stored: An attribute whose value is directly saved in the database and does not depend on other attributes.

Ex: Date of Birth.

Derived: An attribute whose value is calculated or derived from the value of related attribute and it is not permanently stored in the database.

Ex: Age from DOB ($\text{Age} = \text{current date} - \text{Date of birth}$)

Null attributes :

Represents a value that is unknown or not applicable for the

entity.

Ex: A student's ID is missing or not recorded.

Complex Attributes :

An attribute that is both multi-valued and composite.

Ex: Work Experience - A person can have multiple jobs before and each job consists of job name, start date and end date.

Key Attributes :

candidate key: The minimal set of attributes that uniquely identifies the each occurrence of an entity.

Ex: Register Number.

Primary key: The candidate key used to uniquely identify each entity occurrence within an entity set.

Composite key: A candidate key / primary key that consists of two or more attributes.

Ex: Combination of property Number and Rental data uniquely identifies the Rental contract.

Relationship:

→ How two entities are associated with each other is called Relationship.

→ A relationship is a meaningful association or connection among entities.

Degrees:

The degree is the number of entity types participating in that relationship.

i. Unary / Recursive:

single entity type related to itself.

Ex: Staff Supervises staff.

ii. Binary Relationship:

Association between two entity types.

Ex: Staff manages branch.

iii. Ternary Relationship:

Association among three entity types simultaneously.

Ex: staff registers client at a branch.

iv. n-ary relationship:

Association involving n-entity types in the relationship.

Ex: Complex enterprise interactions.

Cardinality Ratio:

The Cardinality Ratio specifies the maximum number of relationship instances that an entity can participate in.

i. One to one (1:1)

ii. One to Many (1:N)

iii. Many to One (N:1)

iv. Many to Many (M:N)

i. One-to-one (1:1) :

An entity in set A is associated with at most one entity in set B and the entity in set B is associated with at most one entity in set A.

Ex: Student owns Register no.

One student owns one Register no.

Register no. is owned by only one student.

ii. one-to-many (1:N) :

An entity in set A can be associated

with any number of entities in set B but an entity in set B is associated with atmost one entity in set A.

Ex: A branch has many employees but each employee works at one branch.

iii. Many-to-one (N:1) :

An entity in set A is associated with atmost one entity in set B while an entity in set B is associated with any number of entities in set A.

Ex: Staffs choosing Department.

Many staffs choose one department

Each staff member belongs to the

one department.

iv. Many - to - many (M:N):

An entity in set A is associated with any number of entities in set B, also Entities in Set B is associated with many number of entities in set A.

Ex: Staffs works on project.

One staff works on many project.

One project is done by many staffs.

participation constraints:

Determines whether an entity participate in the relationship is mandatory or Optional.

- i. Total / Mandatory: Every instance of an entity set must participate in the relationship.

- ii. **partial / optional**: Some instances of an entity set may not participate in the relationship.

Structural constraints :

Structural Constraints are the main restrictions placed on how entities relate to each other. They determine the overall structure of a relationship.

The structural constraints made of two components :

- i. **cardinality ratio** (Maximum limit)
- ii. **participation constraint** (Minimum limit)

Attributes On Relationship type :

Attributes can describe the interaction itself when the property cannot be assigned to either entity individually.

Ex: Staff works_on project

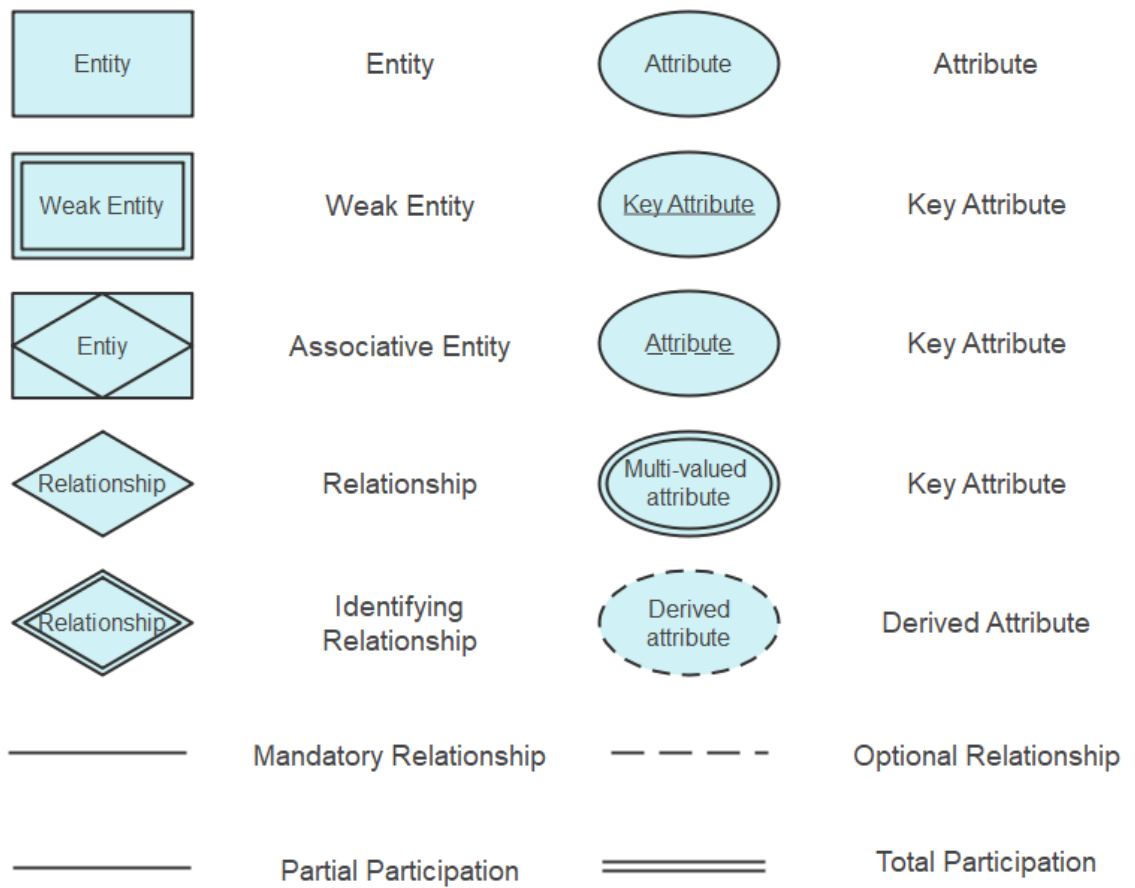
Hours_worked belongs to the work_on relationship itself. (it depends on which Staff and which project).

Weak Entity - Set:

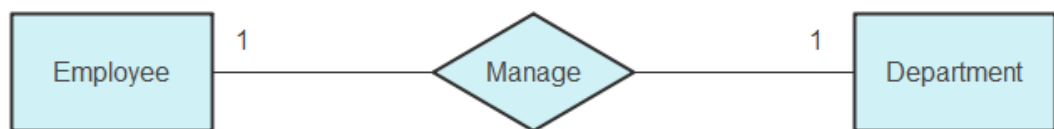
A weak entity set is an entity type whose instances cannot be uniquely identified by their own attributes alone.

Ex: A college may have students with same name taking same course.

Reg.no separates them.



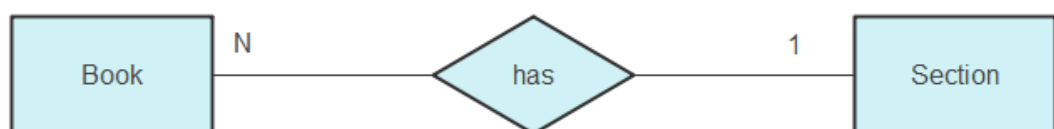
one-to-one (1:1)



one-to-many (1:N)



many-to-one (N:1)



many-to-many (M:N)



Mapping ER model to Relational Schema:

i. Mapping of Regular Entity types :

- Create a table for the Entity.
 - Entity name becomes table name.
 - Single value attribute becomes column name.
 - Components of Composite attribute becomes Column name.
 - Key attribute becomes primary key.
- Derived Attribute can be ignored.

Staff (StaffNo, Name, position, salary)

ii. Mapping of weak Entity sets :

- Create a table for each weak entity.

- Include all simple attributes.
- Weak entity must have foreign key.
- The primary key is formed by combining primary key of owner entity (FK) and the partial key of weak entity.

course	Section
ID (PK)	ID
Name	Name
credits	course_ID (FK)

iii. Mapping of Binary 1:1 Relationship:

- Mandatory participation on both sides.
(1:1) combine both entities into a single relation.
- Mandatory on one side, optional on other.

→ Optional participation on both sides.

→ Left side should have Foreign key.

iv. Mapping of Binary 1:N relationship:

→ Identify the entity on the "one" side (parent) and the entity on the "many" side (child).

→ primary key on the parent entity and the foreign key on the child entity.

→ Right side have foreign key.

propertyForRent (No, street, city, pincode, rooms, person_id^{Fk})

v. Mapping of Binary M:N Relationship:

→ Create a new junction table to represent relationship.

→ Primary key is the composite key consisting of primary keys of both participating entities.

Client (clientNo, ...) and Rent (propertyNo, ...)
 viewing (clientNo^{FK}, propertyNo^{FK}, viewdate)

vi. Mapping of multivalued attributes :

- create a new relation for any multi-valued attribute.
- Include the multivalued attribute column alongside the primary key of the primary key (acting as foreign key)
- primary key is composite of both columns.

Telephone (branchNO^{FK}, teINO)

vii. Mapping of N-ary Relationship:

- For relationship involving more than two entities, create a dedicated relation.
- primary key is the composite of the primary keys of all participating entities.
- Include any attributes that belong to the relationships.

Domain :

A domain is the set of allowable, atomic values for one / more attributes. It defines the datatype, Range, values.

Ex: A domain named "phone number" might be defined as the string of 10 positive digit

Attribute:

An Attribute is a named column in a relation that describes a specific property of the Entity.

Ex: Firstname, lastname, Salary.

Tuple :

A tuple is a single row in a relation representing a discrete record or instance of an entity.

Relational Schema:

A Relational schema is the logical

Structure or definition of a relation.

It is Static, rarely changes.

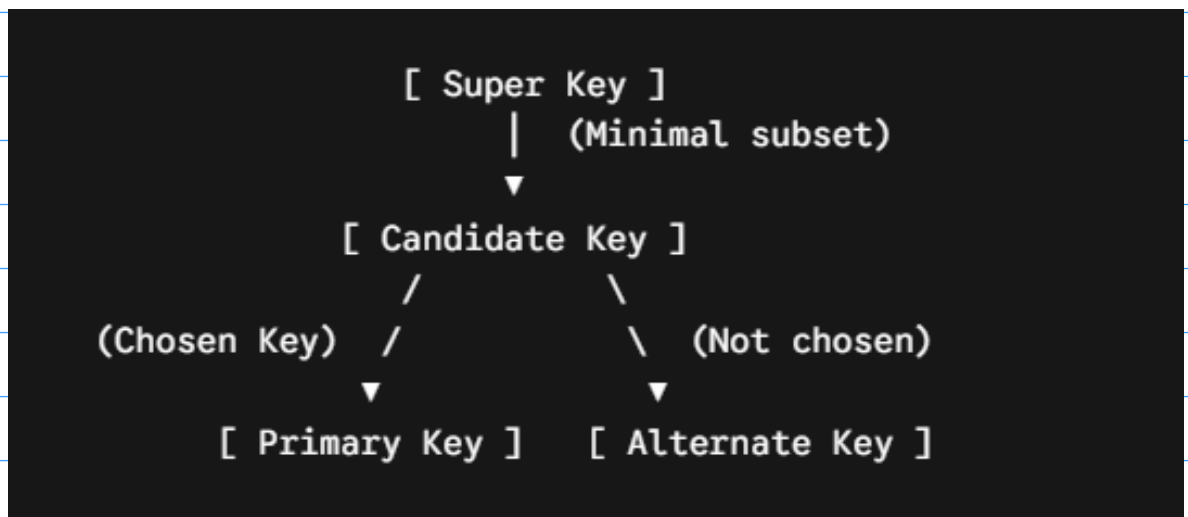
Ex: Student (Name, Reg.NO, DOB)

Relational Instance / State:

A Relational Instance / State is the actual set of Tuples contained in the table at a specific point in time. It is dynamic, the data changes as it is changed, inserted, updated.

Relational keys:

A key consists of one or more attributes whose values uniquely identify each tuple within a relation.



Super key: An Attribute or Set of attribute that uniquely identifies a tuple within a relation. It may contain redundant attributes.

Candidate key: A minimal Super key - meaning no proper subset of its attributes can uniquely identify a tuple.

Primary key: The candidate key selected by the database designer to uniquely

Identify tuples within the relation.

Alternate key : Any candidate key that was not chosen by the primary key.

Foreign key : An Attribute or set of attribute in one table that matches the candidate key of another table, that is used to establish link between relation.

Integrity - constraints :

Integrity constraints are rules that ensure data accuracy, consistency and Validity across updates.

Domain constraints :

Requires that every value entered

into an attribute must belong to the permitted values that is specified by the attributes domain.

Ex! Data types, Range, Values.

Entity Integration:

In a base relation, no attributes of a primary key can be NULL. Primary keys are used to uniquely identify individual types. If the primary key is NULL, then the system could not distinguish between records.

Referential Integrity:

If a foreign key exists in a relation the foreign key value must either match a

Valid candidate key value in its home relation or be entirely Null. It prevents dangling references (Assigning an employee to branch id that doesn't exist in Branch table)

The rules followed by Referential Integrity:

- i. It cannot update / change the primary key in the primary table.
- ii. It cannot delete primary key in the primary table.
- iii. It cannot insert Foreign key that is not present as primary key in the primary table.

iv. It can insert NULL value as foreign key.

Enterprise / Business constraints :

Additional business logic rules

Specified by users or DBA who govern the organization policies.

Ex: An employee cannot earn more than 15 lakhs per month without the approval of organization or company.

Cascade update :

When a primary key is updated / modified in the primary table, DBMS automatically updates the matching foreign key values in all child rows.

Ex: changing BranchNo automatically updates BranchNo assigned to Staff rows.

Cascade delete:

When a row is deleted from the parent table, DBMS automatically deletes all the matching rows in the child table.

Ex: Deleting order automatically deletes all corresponding orderItem rows.

Schema Refinement :

Schema Refinement is the process of decomposing bad or unnormalized relation Schemas into smaller, well-structured relations to improve database efficiency, Consistency and quality.

Informal Design Guidelines :

i. Semantic Of the Attributes :

Design relations so that the meaning is straight-forward to explain. The goal is to ensure represent exactly one entity or relationship.

Bad EX: Mixing Department and Managers into a single table.

ii. Reducing Redundant Values in Tuple :

Base Values should be designed so that to avoid storing redundant value which leads to update anomalies.

Insertion Anomaly : Unable to insert a record for a new entity without filling

Unneeded attributes in the table.

Ex: Adding a new department requires adding a dummy employee.

Deletion Anomaly: Deleting a row accidentally deletes information about a completely different entity.

Ex: Deleting the last employee in the department causes to lose all the information about the department and question its existence.

Modification/update Anomaly: updating a single piece of information requires updating multiple rows leading to data inconsistency if any row is missed.

The goal is to minimize the storage waste and maintain consistency during updates.

iii. Reducing NULL Values in Tuples :

To minimize the attributes whose values may frequently be NULL. NULL values waste storage, create ambiguity and create unpredictable results when joining two tables.

The solution is to move those attributes to a separate table linked by primary key / Foreign key.

iv. Disallow Spurious Tuples :

The goal is to guarantee the lossless-joining property during schema refinement.

It is to design relations such that joining them on matching key produces only valid original data and no fake rows (Spurious tuples)

Guideline	What to Avoid	Solution
1. Semantics	Mixing multiple entities into one table	Keep 1 entity/relationship per table
2. Anomalies	Duplicate data, update/insert/delete errors	Separate redundant attributes via normalization
3. Null Values	Tables with sparse/empty columns	Split optional attributes into sub-tables
4. Spurious Tuples	Fake records appearing after a NATURAL JOIN	Join tables ONLY on Foreign Key → Primary Key pairs

Closure Algorithm :

This algorithm is used to find candidate keys, primary keys, alternative keys, composite keys.

⇒ Find the primary key from the Given relations with functional dependencies.

$$A \rightarrow BC$$

$$B \rightarrow C$$

$$C \rightarrow D$$

$$D \rightarrow E$$

$$A^+ = \{A, B, C, D, E\}$$

$$B^+ = \{B, C, D, E\}$$

$$C^+ = \{C, D, E\}$$

$$D^+ = \{D, E\}$$

∴ The set which contains all attributes is the primary key.

A is the primary key.

⇒ Find the primary key from the Given relations with functional dependencies.

$$A \rightarrow BC$$

$$B \rightarrow D$$

$$C \rightarrow A$$

$$E \rightarrow B$$

$$A^+ = \{A, B, C, D\}$$

$$B^+ = \{B, D\}$$

$$C^+ = \{C, A, B, D\}$$

$$E^+ = \{E, B\}$$

$\therefore$ A and C contain the same attributes. So
Either one should combine with other set
so that it becomes primary key.

Let's choose A and E, C and E also can
be choosed, so

$$(AE)^+ = \{A, B, C, D, E\}$$

Since more than one attribute is required to uniquely identify the entity, the primary key here is the composite key.